Protein contrasts

Table of contents

Packages 1
Inputs 1
The fit 2
Results per protein and contrast 3
Is each test calibrated? 4
Do one group's responders move in the other group? 7
Every protein at a glance 9
Set trajectories 13
Whole-proteome structure 14
Validation 17
Summary 17
Save 18

Every protein tested in each of the nine contrasts from step 01. The fit is limma on limpa's values, with limpa's standard errors as precision weights, person as a block and blood as a covariate. Each effect is a log2 fold change (logFC: +1 means twice as much) with its 95% confidence interval.

Packages

```
library(here)
library(limma)
library(dplyr)
library(tidyr)
library(purrr)
library(tibble)
library(ggplot2)
library(grid)
library(ComplexHeatmap)
source(here("00_Setup", "Shared_functions", "analysis_reporting_helpers.R"))
params$grouping
```

```
[1] "Proteome_T2"
```

Inputs

`01_design.rds` holds this grouping's design, contrasts and people. `proteins.rds` holds limpa's values and standard errors; `DAList_corrected.rds` the same values with blood slopes removed, used here only for the PCA and the set trajectories.

```
data_dir <- file.path("03_Group_comparisons", params$grouping, "c_data")
inputs <- c(
```

```
  design = file.path(data_dir, "01_design.rds"),
  proteins = "01_Processing/03_Quantification/c_data/proteins.rds",
  corrected = "01_Processing/03_Quantification/c_data/DAList_corrected.rds",
  phenotype = "00_Setup/Input/phenotype.csv"
)
paths <- map_chr(inputs, here)
stopifnot(file.exists(paths))
d <- readRDS(paths[["design"]])
arms <- d$arms
dal <- d$dal
contrast_names <- colnames(dal$design$contrast_matrix)
stopifnot(identical(rownames(dal$design$design_matrix), colnames(dal$data)))
proteins <- subset_for_design(readRDS(paths[["proteins"]]), d)$proteins
design <- dal$design$design_matrix
contrast_matrix <- dal$design$contrast_matrix
meta <- dal$metadata
stopifnot(
  identical(colnames(proteins$E), rownames(design)), identical(meta$sample_id,
rownames(design))
)
dim(proteins$E)
```

```
[1] 1900   44
```

The fit

`fit_limpa_contrasts()` (shared helper) runs limpa's `dpcDE()`: limma's linear model with each value weighted by the inverse of its limpa standard error, so a value the detection curve had to estimate counts less than a measured one, and with person as a block. It then applies the nine contrasts and empirical Bayes moderation (`eBayes(robust = TRUE)`). Moderation borrows variance information across proteins, which steadies each protein's own variance estimate at n = 8 per group; the robust version stops a few hypervariable proteins from distorting that borrowing (Phipson et al. 2016).

```
eb <- fit_limpa_contrasts(proteins, design, contrast_matrix, meta$subject)
summary(eb$df.total)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  41.49   42.49   42.49   42.49   42.49   42.49
```

The same nine contrasts without the blood term show how much the blood covariate moves each estimate. They are saved for comparison and decide nothing.

```
blood_free <- design[, colnames(design) != "blood"]
raw_eb <- fit_limpa_contrasts(
  proteins, blood_free, contrast_matrix[colnames(blood_free), ], meta$subject
)
unadjusted <- tibble(
  uniprot_id = rep(rownames(proteins$E), length(contrast_names)),
```

```
  contrast = rep(contrast_names, each = nrow(proteins$E)),
  logFC = c(raw_eb$coefficients[, contrast_names]), p = c(raw_eb$p.value[, contrast_names])
) |>
  pivot_wider(names_from = contrast, values_from = c(logFC, p), names_glue = "{contrast}
_{.value}")
```

Results per protein and contrast

`topTable()` gives each protein's logFC, its 95% confidence interval and the moderated t and p. Three corrections sit beside the nominal p, each within one contrast:

- `fdr`, Benjamini-Hochberg: the expected share of false calls among proteins called at that level;
- `qvalue`, Storey's q (`storey_q()`): BH scaled by the estimated share of proteins with no effect;
- `pi_score`, Π = p^|logFC| (Xiao et al. 2014): small when p is small and the fold change large. Π < 0.05 currently only selects the protein sets for the fry check below.

limpa gives every protein a value in every run, so every contrast tests every protein.

```
annotation <- as_tibble(dal$annotation)[, c("uniprot_id", "gene", "protein", "description")]
stopifnot(identical(rownames(eb$coefficients), annotation$uniprot_id))
results <- map(set_names(contrast_names), \(contrast) {
  topTable(eb, coef = contrast, number = Inf, sort.by = "none", confint = TRUE) |>
    as_tibble() |>
    select(logFC, logFC_lower = CI.L, logFC_upper = CI.R, AveExpr, t, P.Value, adj.P.Val) |>
    bind_cols(annotation, x = _)
}) |>
  list_rbind(names_to = "contrast") |>
  rename(p = P.Value, fdr = adj.P.Val) |>
  mutate(qvalue = storey_q(p), .by = contrast) |>
  mutate(
    pi_score = p^abs(logFC),
    sig_pi = case_when(
      pi_score < 0.05 & logFC > 0 ~ 1L,
      pi_score < 0.05 & logFC < 0 ~ -1L,
      TRUE ~ 0L
    )
  )
n_obs <- set_names(rowSums(proteins$other$n.observations > 0), rownames(proteins$E))
results |>
  filter(contrast == "Training_Interaction") |>
  slice_min(p, n = 5) |>
  select(gene, logFC, logFC_lower, logFC_upper, p, fdr)
```

```
# A tibble: 5 × 6
  gene   logFC logFC_lower logFC_upper         p    fdr
  <chr>  <dbl>       <dbl>       <dbl>     <dbl>  <dbl>
1 GCN1    2.02       1.15        2.89  0.0000297 0.0564
2 MRPL49  1.34       0.712       1.96  0.0000930 0.0652
3 LRPAP1 -1.68      -2.48       -0.891 0.000103  0.0652
```

```
4 BCKDHB  2.09       1.06        3.12  0.000195  0.0923
5 RAB14  -1.06      -1.60       -0.529 0.000243  0.0923
```

```
contrast_summary <- results |>
  summarise(
    n_tested = n(), n_nominal = sum(p < 0.05), n_fdr_05 = sum(fdr < 0.05),
    n_fdr_10 = sum(fdr < 0.10), min_fdr = min(fdr),
    .by = contrast
  ) |>
  left_join(d$roles, by = "contrast")
contrast_summary
```

```
# A tibble: 9 × 7
  contrast             n_tested n_nominal n_fdr_05 n_fdr_10 min_fdr role
  <chr>                   <int>     <int>    <int>    <int>   <dbl> <chr>
1 Training_C1              1900       112        1        3  0.0157 descriptive
2 Training_C2              1900       268        0        8  0.0755 descriptive
3 Acute_C1                 1900       265        1        3  0.0155 descriptive
4 Acute_C2                 1900       152        0        0  0.222  descriptive
5 Baseline_C1vC2           1900       178        0        0  0.487  floor
6 Trained_C1vC2            1900       247        0        1  0.0857 descriptive
7 Acute_C1vC2              1900        52        0        0  0.993  descriptive
8 Training_Interaction     1900       176        0        5  0.0564 primary
9 Acute_Interaction        1900       244        2        7  0.0456 secondary
```

Is each test calibrated?

Under no effect at all, p values are flat, median |t| is about 0.674 and 20% of p fall below 0.2. limma's `propTrueNull()` saturates at 1, so it cannot tell a flat null from a conservative test; median |t| and the share of p below 0.2 can.

```
null_calibration <- results |>
  summarise(
    prop_true_null = round(propTrueNull(p), 3),
    median_abs_t = round(median(abs(t)), 3),
    p_below_0.2 = round(mean(p < 0.2), 3), .by = contrast
  )
null_calibration
```

```
# A tibble: 9 × 4
  contrast             prop_true_null median_abs_t p_below_0.2
  <chr>                         <dbl>        <dbl>       <dbl>
1 Training_C1                   0.99         0.667       0.223
2 Training_C2                   0.781        0.96        0.367
3 Acute_C1                      0.807        0.943       0.354
4 Acute_C2                      0.9          0.813       0.268
5 Baseline_C1vC2                0.899        0.802       0.271
6 Trained_C1vC2                 0.848        0.87        0.318
```

```
7 Acute_C1vC2                  1           0.62        0.149
8 Training_Interaction         0.907       0.787       0.275
9 Acute_Interaction            0.822       0.901       0.332
```

The smallest effect the primary contrast could detect with 80% power at a two-sided 0.05, from each protein's moderated standard error:

```
se <- eb$stdev.unscaled[, "Training_Interaction"] * sqrt(eb$s2.post)
detectable <- se * (qt(0.975, eb$df.total) + qt(0.80, eb$df.total))
quantile(detectable, c(0.5, 0.9), na.rm = TRUE)
```

```
      50%       90%
0.9334119 1.4347464
```

```
results |>
  mutate(contrast = factor(contrast, levels = contrast_names)) |>
  ggplot(aes(p)) +
  geom_histogram(binwidth = 0.05, boundary = 0, fill = "grey35") +
  geom_hline(
    aes(yintercept = n_tested / 20),
    mutate(contrast_summary, contrast = factor(contrast, levels = contrast_names)),
    linetype = 2, colour = "firebrick"
  ) +
  facet_wrap(~contrast) +
  labs(x = "p", y = "proteins") +
  theme_minimal(base_size = 10)
```

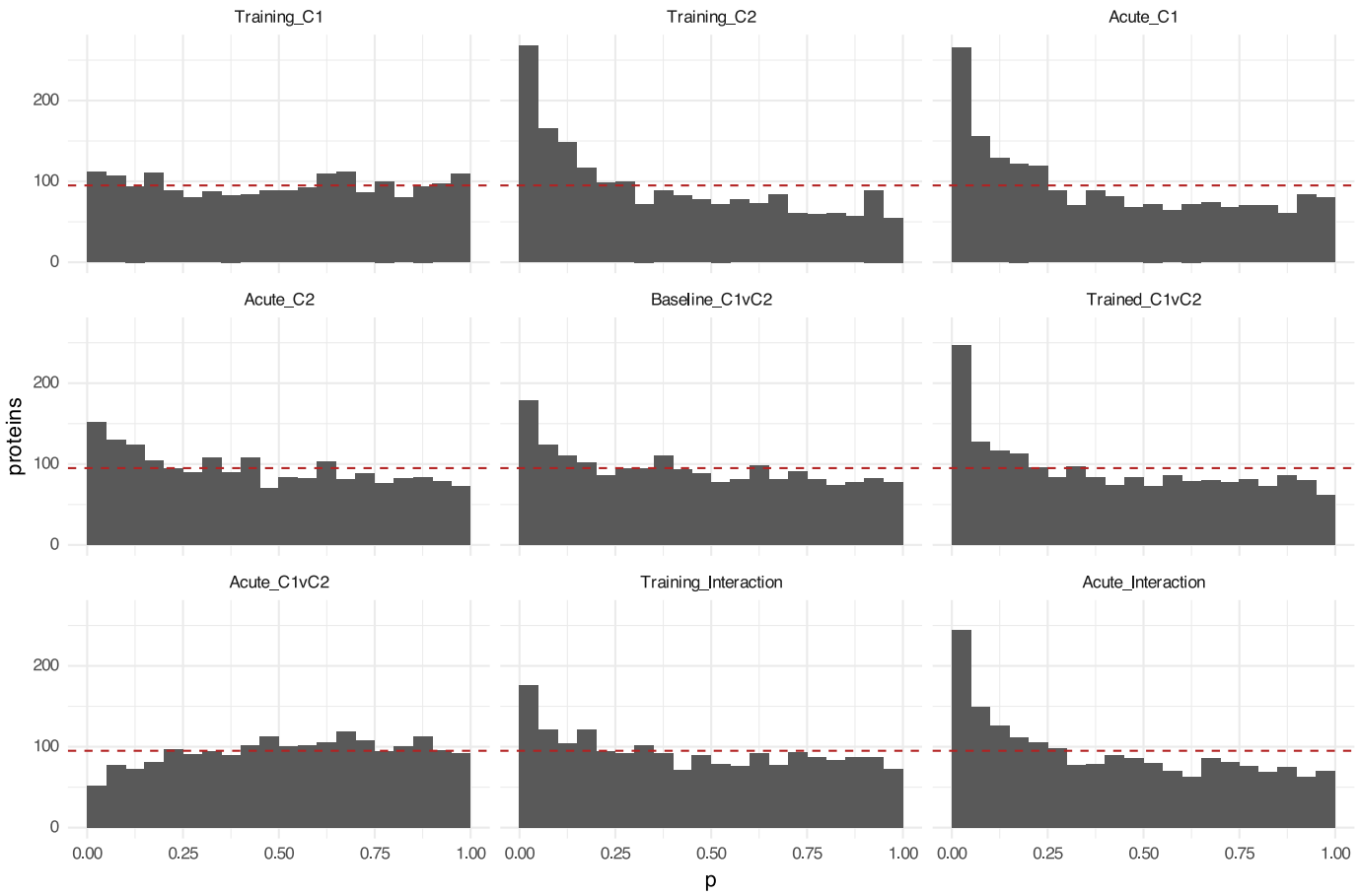


Figure 1: p values per contrast. Dashed: a flat null (tested / 20).

A spike at zero on a flat background is signal; a slope up towards 1 is a conservative test.

```
results |>
  mutate(
    contrast = factor(contrast, levels = contrast_names),
    call = case_when(fdr < 0.05 ~ "BH < 0.05", p < 0.05 ~ "p < 0.05", TRUE ~ "not nominal")
  ) |>
  ggplot(aes(logFC, -log10(p), colour = call)) +
  geom_point(size = 0.6, alpha = 0.7) +
  geom_hline(yintercept = -log10(0.05), linetype = 2, colour = "grey50") +
  ggrepel::geom_text_repel(
    aes(label = gene),
    data = \(x) filter(x, fdr < 0.05), size = 2.2, show.legend = FALSE
  ) +
  facet_wrap(~contrast) +
  scale_colour_manual(
    values = c(`BH < 0.05` = "#B2182B", `p < 0.05` = "#2166AC", `not nominal` = "grey75"),
    name = NULL
  ) +
  labs(x = "log2 fold change", y = "-log10 p") +
```

```
  theme_minimal(base_size = 9) +
  theme(legend.position = "top")
```

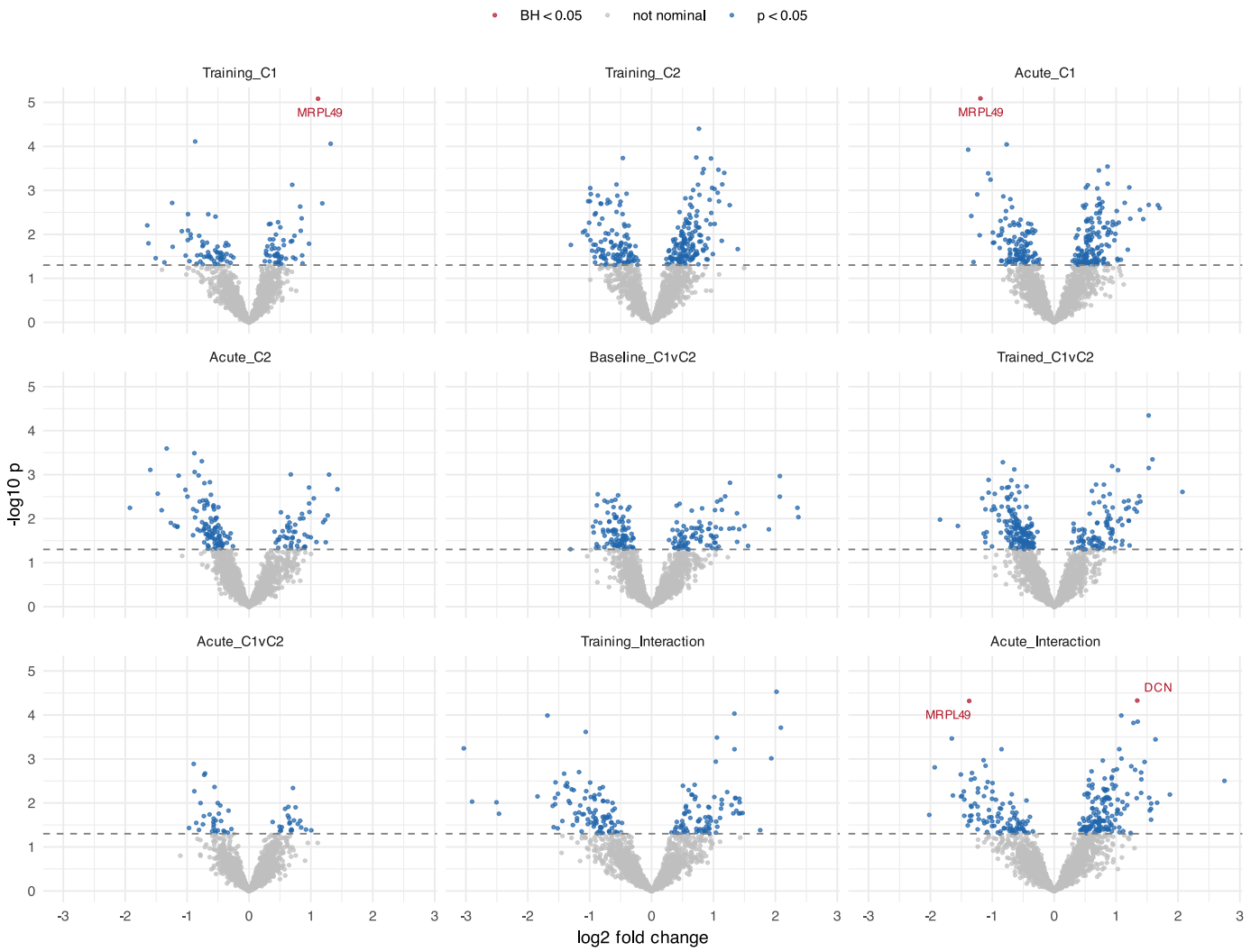


Figure 2: Volcano per contrast. Dashed: p = 0.05. Labels: BH < 0.05.

Do one group's responders move in the other group?

Proteins one group changes at p < 0.05 are partly chance, and their fold change in that group is inflated by the selection. Testing the same proteins in the other group's contrast, which uses different people, asks whether the change carries over. fry (limma) tests a whole set at once, treating people as the unit. Sets come from three criteria (Π, BH, nominal) and two directions, and need at least 5 proteins.

Only the other-group pairings share no cell mean. A training set tested on its own group's acute contrast shares T2, so a protein high at T2 by chance reads as up then down; those rows are reported but carry that caution. fry reads limpa's complete matrix with the same design and block; the within-person correlation comes from `duplicateCorrelation()`.

```
abundance <- proteins$E
within_subject <- duplicateCorrelation(abundance, design, block = meta$subject)
fry_correlation <- within_subject$consensus.correlation
member_rows <- function(cn, criterion, direction) {
  r <- filter(results, contrast == cn)
  sign_ok <- if (direction == "up") r$logFC > 0 else r$logFC < 0
  hit <- switch(criterion,
    pi = r$sig_pi == if (direction == "up") 1L else -1L,
    bh = r$fdr < 0.05 & sign_ok,
    nominal = r$p < 0.05 & sign_ok
  )
  which(rownames(abundance) %in% r$uniprot_id[hit])
}
training <- paste0("Training_", arms)
acute <- paste0("Acute_", arms)
pairings <- tribble(
  ~set_from, ~ranked_on,
  training[1], training[2],
  training[2], training[1],
  acute[1], acute[2],
  acute[2], acute[1],
  training[1], acute[1],
  training[1], acute[2],
  training[2], acute[2],
  training[2], acute[1]
)
selections <- expand_grid(criterion = c("pi", "bh", "nominal"), direction = c("up", "down"))
```

For each pairing, `fry()` gets every selected set of 5 or more proteins. Beside each test sits the set's mean logFC in the group that chose it (inflated by the choice) and in the other contrast, and the share of members whose sign carries over. fry drops its FDR column when handed a single set, so a pairing needs at least two sets.

```
fry_concordance <- pmap(pairings, \(set_from, ranked_on) {
  sets <- pmap(selections, \(criterion, direction) member_rows(set_from, criterion, direction))
|>
    set_names(paste(selections$criterion, selections$direction, sep = "_"))
  sets <- keep(sets, \(rows) length(rows) >= 5)
  if (length(sets) < 2) {
    return(NULL)
  }
  log_fc <- \(cn, rows) filter(results, contrast == cn)$logFC[rows]
  carried <- imap(sets, \(rows, set) {
    own <- log_fc(set_from, rows)
    other <- log_fc(ranked_on, rows)
    tibble(
      set,
      mean_logfc_own = mean(own), mean_logfc_other = mean(other),
      same_sign_share = mean(sign(own) == sign(other))
    )
  }) |>
```

```
    list_rbind()
  fry(abundance,
    index = sets, design = design, contrast = contrast_matrix[, ranked_on],
    block = meta$subject, correlation = fry_correlation
  ) |>
    as_tibble(rownames = "set") |>
    transmute(
      set_from, ranked_on, set,
      n_proteins = NGenes, direction = Direction, p = PValue, fdr = FDR,
      p_mixed = PValue.Mixed, fdr_mixed = FDR.Mixed
    ) |>
    left_join(carried, by = "set")
}) |>
  list_rbind()
fry_concordance |>
  filter(set == "nominal_up" | set == "nominal_down") |>
  select(set_from, ranked_on, set, n_proteins, direction, p, same_sign_share)
```

```
# A tibble: 16 × 7
   set_from    ranked_on   set      n_proteins direction       p same_sign_share
   <chr>       <chr>       <chr>         <int> <chr>       <dbl>           <dbl>
 1 Training_C1 Training_C2 nominal…         53 Up        8.22e-2          0.642
 2 Training_C1 Training_C2 nominal…         59 Up        4.57e-1          0.492
 3 Training_C2 Training_C1 nominal…        156 Up        3.79e-1          0.628
 4 Training_C2 Training_C1 nominal…        112 Up        8.46e-1          0.455
 5 Acute_C1    Acute_C2    nominal…        116 Down      4.66e-1          0.603
 6 Acute_C1    Acute_C2    nominal…        149 Up        7.41e-1          0.557
 7 Acute_C2    Acute_C1    nominal…         50 Up        5.77e-1          0.52
 8 Acute_C2    Acute_C1    nominal…        102 Down      9.32e-1          0.5
 9 Training_C1 Acute_C1    nominal…         53 Down      3.68e-7          0
10 Training_C1 Acute_C1    nominal…         59 Up        1.11e-4          0.102
11 Training_C1 Acute_C2    nominal…         53 Down      2.58e-1          0.396
12 Training_C1 Acute_C2    nominal…         59 Down      8.72e-1          0.508
13 Training_C2 Acute_C2    nominal…        156 Down      1.48e-4          0.0897
14 Training_C2 Acute_C2    nominal…        112 Up        1.50e-2          0.0893
15 Training_C2 Acute_C1    nominal…        112 Down      4.59e-1          0.625
16 Training_C2 Acute_C1    nominal…        156 Down      5.17e-1          0.372
```

Every protein at a glance

Two heatmaps: every protein across every biopsy (centred on the protein's mean), then across the nine contrasts. Rows follow average-linkage clustering on 1 − Pearson r of limpa's matrix, for display only. Colours stop at the 99th percentile of |value| (`cap()`). Trait tracks sit on the T1 and T2 columns; T3 has no phenotype.

```
sample_values <- proteins$E
feature_label <- set_names(annotation$gene, annotation$uniprot_id)
contrast_results <- transmute(results, feature = uniprot_id, contrast, effect = logFC, p, fdr)
row_groups <- list(
  proteins = rownames(abundance)[hclust(as.dist(1 - cor(t(abundance))), "average")$order]
```

```
)
report_arm_colours <- set_names(c("#2166AC", "#B2182B"), arms)
trait_table <- readr::read_csv(paths[["phenotype"]], show_col_types = FALSE)
biopsy_traits <- trait_table |>
  select(subject, matches("_(pre|post)_")) |>
  pivot_longer(-subject, names_to = c("trait", "when"), names_pattern = "(.*)_(pre|post)_.*") |
>
  pivot_wider(names_from = trait) |>
  transmute(
    subject,
    timepoint = if_else(when == "pre", "T1", "T2"), fcsa_mixed, mcsa, leg_press_1rm
  )
columns <- as_tibble(meta) |>
  transmute(sample_id, subject, arm = as.character(arm), timepoint = as.character(timepoint)) |
>
  mutate(group = factor(
    paste(arm, timepoint, sep = "_"),
    paste(rep(arms, each = 3), c("T1", "T2", "T3"), sep = "_")
  )) |>
  left_join(biopsy_traits, by = c("subject", "timepoint")) |>
  left_join(select(trait_table, subject, comp_hypertrophy), by = "subject") |>
  arrange(group, subject)
stopifnot(!anyDuplicated(columns$sample_id), setequal(columns$sample_id,
colnames(sample_values)))
```

```
grey_ramp <- \(x) circlize::colorRamp2(range(x, na.rm = TRUE), c("#F7F7F7", "#252525"))
tracks <- list(
  "fCSA mixed (um2)" = columns$fcsa_mixed,
  "mCSA (cm2)" = columns$mcsa, "leg press 1RM (kg)" = columns$leg_press_1rm,
  "comp_hypertrophy, per subject" = columns$comp_hypertrophy
)
sample_annotation <- HeatmapAnnotation(
  df = data.frame(arm = columns$arm, timepoint = columns$timepoint, tracks, check.names =
FALSE),
  col = c(
    list(arm = report_arm_colours, timepoint = c(T1 = "grey85", T2 = "grey55", T3 = "grey25")),
    map(tracks, grey_ramp)
  ),
  na_col = "grey80", annotation_name_gp = gpar(fontsize = 7), annotation_name_side = "left",
  simple_anno_size = unit(3, "mm")
)
effect_limit <- cap(contrast_results$effect)
overviews <- imap(row_groups, \(rows, group) {
  m <- sample_values[rows, columns$sample_id, drop = FALSE]
  m <- sweep(m, 1, rowMeans(m, na.rm = TRUE))
  limit <- cap(m)
  labelled_heat <- \(...) make_report_heatmap(..., labels = feature_label)
  biopsy_map <- labelled_heat(m, "biopsies", c(-limit, 0, limit), "log2 abundance, centred",
    row_names = TRUE, column_title = character(0), top_annotation = sample_annotation,
    column_split = columns$group, cluster_column_slices = FALSE,
```

```
    column_labels = columns$subject, column_names_gp = gpar(fontsize = 5)
  )
  contrast_grid <- \(value) {
    results_to_matrix(contrast_results, rows, contrast_names, "contrast", value)
  }
  contrast_maps <- labelled_heat(
    contrast_grid(\(d) d$effect), "log2 fold change", c(-effect_limit, 0, effect_limit), "log2
FC"
  ) +
    labelled_heat(contrast_grid(\(d) status_of(d$p, d$fdr)), "moderated t p and BH q",
      row_names = TRUE
    )
  list(
    heatmap_page(biopsy_map, sprintf("Biopsy overview: %s", group), paste(
      sprintf("All %s proteins by %d biopsies.", format(length(rows), big.mark = ","),
ncol(m)),
      sprintf("Centred log2 abundance; colours stop at %.2g.", limit)
    ), legends = list()),
    heatmap_page(contrast_maps, sprintf("Contrast overview: %s", group), paste(
      "Log2 fold change from the limma fit; status: p and BH q within contrast.",
      sprintf("Colours stop at %.2g.", effect_limit)
    ))
  )
}) |>
  list_flatten()
walk(overviews, \(page) page$draw())
```

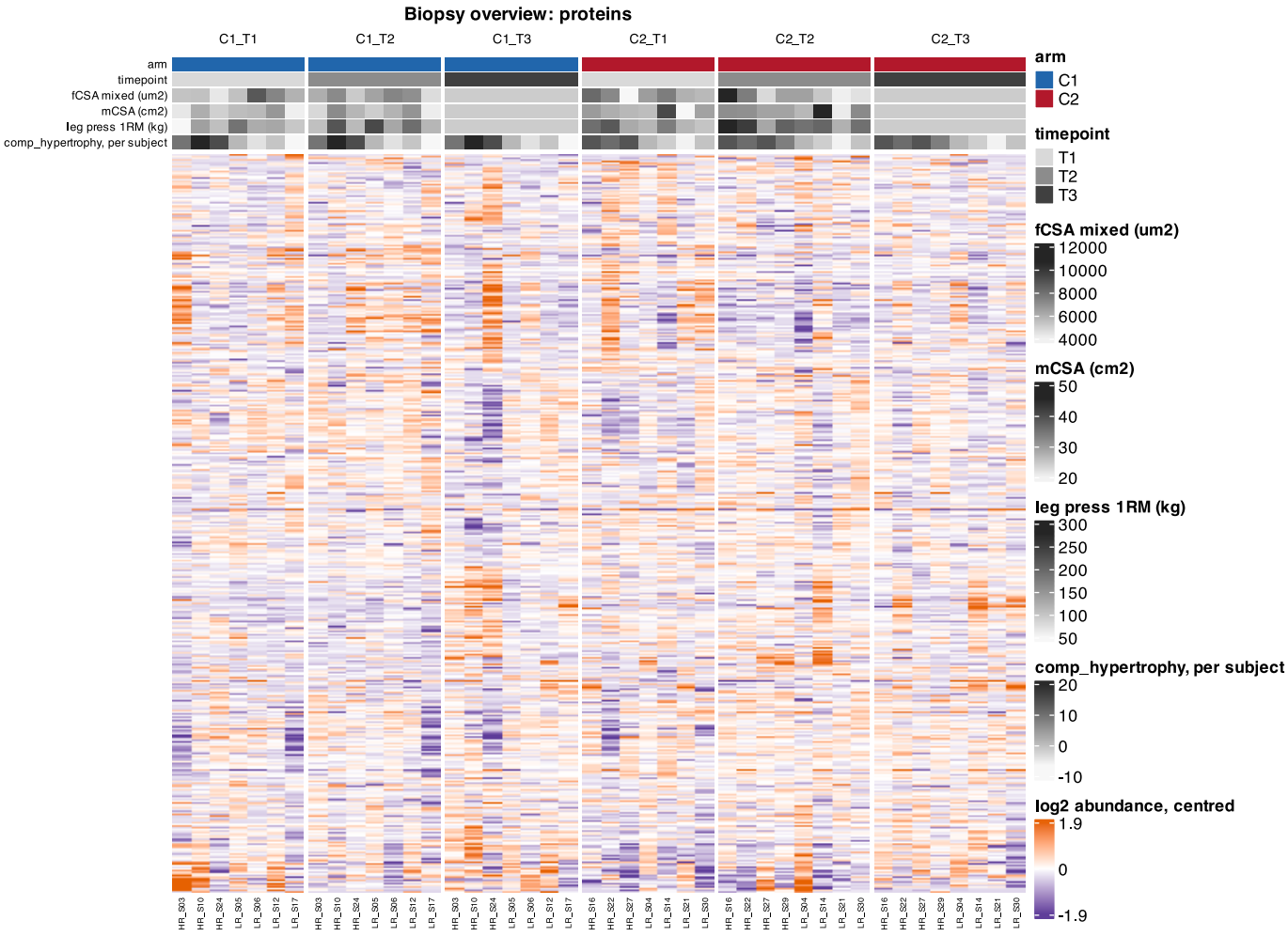


Figure 3: Every protein across biopsies, then across the nine contrasts.



Figure 4: Every protein across biopsies, then across the nine contrasts.

Set trajectories

Each group's p < 0.05 training sets (up and down), followed through T1, T2 and T3 in both groups: each person's set mean on the blood-corrected matrix minus their own T1 value, then the mean and standard error across people. A person with no T1 biopsy drops out.

```
corrected <- subset_for_design(readRDS(paths[["corrected"]]), d)
sample_meta <- as_tibble(meta)
stopifnot(identical(rownames(corrected$data), rownames(abundance)))
set_trajectory <- results |>
  filter(contrast %in% training, p < 0.05) |>
  mutate(set_from = contrast, set = if_else(logFC > 0, "nominal_up", "nominal_down")) |>
  summarise(members = list(uniprot_id), .by = c(set_from, set)) |>
  pmap(\(set_from, set, members) {
    tibble(
      sample_id = colnames(corrected$data),
```

```
      value = colMeans(corrected$data[members, , drop = FALSE])
    ) |>
      left_join(select(sample_meta, sample_id, subject, arm, timepoint), by = "sample_id") |>
      filter("T1" %in% timepoint, .by = subject) |>
      mutate(change = value - value[timepoint == "T1"], .by = subject) |>
      summarise(
        mean_change = mean(change), se = sd(change) / sqrt(n()), n_subjects = n(),
        .by = c(arm, timepoint)
      ) |>
      mutate(set_from = set_from, set = set, n_proteins = length(members), .before = 1)
  }) |>
  list_rbind()
head(set_trajectory, 6)
```

```
# A tibble: 6 × 8
  set_from    set       n_proteins arm   timepoint mean_change     se n_subjects
  <chr>       <chr>          <int> <fct> <fct>           <dbl>  <dbl>      <int>
1 Training_C1 nominal_…         59 C1    T1              0     0                7
2 Training_C1 nominal_…         59 C2    T1              0     0                7
3 Training_C1 nominal_…         59 C1    T2             -0.704 0.127            7
4 Training_C1 nominal_…         59 C2    T2              0.120 0.164            7
5 Training_C1 nominal_…         59 C1    T3             -0.220 0.109            7
6 Training_C1 nominal_…         59 C2    T3              0.197 0.0906           7
```

Whole-proteome structure

A PCA of the blood-corrected matrix (proteins scaled to unit variance), and PERMANOVA (vegan's `adonis2()`, 9,999 permutations) on the Euclidean distances between runs. PERMANOVA asks how much of the distance between runs a factor explains (R^2). Time is tested with permutations kept inside each person, so a person's biopsies are compared with each other. Group is tested on one profile per person (the mean of their runs), or at one time point, so each person counts once.

```
scaled <- scale(t(corrected$data))
pca <- prcomp(scaled)
pca_variance <- tibble(
  component = paste0("PC", 1:4),
  variance_explained = round(summary(pca)$importance[2, 1:4], 3)
)
pca_scores <- as_tibble(pca$x[, 1:2], rownames = "sample_id") |>
  left_join(select(sample_meta, sample_id, subject, arm, timepoint), by = "sample_id")
pca_variance
```

```
# A tibble: 4 × 2
  component variance_explained
  <chr>                  <dbl>
1 PC1                    0.14
2 PC2                    0.093
```

```
3 PC3                    0.082
4 PC4                    0.056
```

`permanova()` takes a row selection, the term to test and whether permutations stay within a person, and returns n, R^2, F and p. It is called once per window and group.

```
permanova <- function(rows, term, within_subject) {
  x <- scaled[rows, , drop = FALSE]
  data <- sample_meta[rows, ]
  if (!within_subject && anyDuplicated(data$subject)) {
    x <- rowsum(x, data$subject) / as.vector(table(data$subject))
    data <- distinct(data, subject, arm)[match(rownames(x), unique(data$subject)), ]
  }
  control <- if (within_subject) {
    permute::how(blocks = factor(data$subject), nperm = 9999)
  } else {
    permute::how(nperm = 9999)
  }
  set.seed(1)
  fit <- vegan::adonis2(dist(x) ~ ., data = data[term], permutations = control)
  tibble(n_samples = nrow(x), r2 = fit$R2[1], f = fit$F[1], p = fit$`Pr(>F)`[1])
}
time_pairs <- list(all = c("T1", "T2", "T3"), training = c("T1", "T2"), acute = c("T2", "T3"))
global_tests <- bind_rows(
  imap(time_pairs, \(times, window) {
    map(c("both", arms), \(who) {
      rows <- sample_meta$timepoint %in% times & (who == "both" | sample_meta$arm == who)
      permanova(rows, "timepoint", within_subject = TRUE) |>
        mutate(term = "time", window = window, arms = who, .before = 1)
    })
  }) |>
    list_flatten() |>
    list_rbind(),
  map(c("subject mean", "T1", "T2", "T3"), \(when) {
    rows <- when == "subject mean" | sample_meta$timepoint == when
    permanova(rows, "arm", within_subject = FALSE) |>
      mutate(term = "arm", window = when, arms = "both", .before = 1)
  }) |>
    list_rbind()
)
```

The same test on the phenotype, the six size and strength traits at T1 and T2 scaled, with time within person, puts the proteome's time effect beside the muscle's own. Group is not tested here: HR/LR and the muscle clusters were split on these traits.

```
trait_sets <- phenotype_tables(trait_table)
trait_rows <- filter(trait_sets$biopsy, if_all(all_of(trait_sets$traits), \(x) !is.na(x)))
set.seed(1)
trait_fit <- vegan::adonis2(
```

```
  dist(scale(trait_rows[trait_sets$traits])) ~ timepoint,
  data = trait_rows,
  permutations = permute::how(blocks = factor(trait_rows$subject), nperm = 9999)
)
global_tests <- global_tests |>
  mutate(data = "proteome", .before = 1) |>
  add_row(
    data = "phenotype", term = "time", window = "all", arms = "both",
    n_samples = nrow(trait_rows), r2 = trait_fit$R2[1], f = trait_fit$F[1],
    p = trait_fit$`Pr(>F)`[1]
  )
global_tests
```

```
# A tibble: 14 × 8
   data      term  window       arms  n_samples     r2     f       p
   <chr>     <chr> <chr>        <chr>     <int>  <dbl> <dbl>   <dbl>
 1 proteome  time  all          both         44 0.0662 1.45  0.0013
 2 proteome  time  all          C1           21 0.117  1.20  0.0295
 3 proteome  time  all          C2           23 0.128  1.47  0.0003
 4 proteome  time  training     both         29 0.0445 1.26  0.0435
 5 proteome  time  training     C1           14 0.0709 0.916 0.375
 6 proteome  time  training     C2           15 0.101  1.46  0.0391
 7 proteome  time  acute        both         30 0.0562 1.67  0.0005
 8 proteome  time  acute        C1           14 0.111  1.50  0.0156
 9 proteome  time  acute        C2           16 0.103  1.61  0.00781
10 proteome  arm   subject mean both         15 0.0642 0.892 0.640
11 proteome  arm   T1           both         14 0.0862 1.13  0.242
12 proteome  arm   T2           both         15 0.116  1.70  0.0012
13 proteome  arm   T3           both         15 0.0588 0.812 0.759
14 phenotype time  all          both         31 0.0355 1.07  0.0372
```

```
ggplot(pca_scores, aes(PC1, PC2, colour = timepoint, shape = arm)) +
  geom_point(size = 2) +
  scale_colour_manual(values = c(T1 = "#E69F00", T2 = "#0072B2", T3 = "#009E73")) +
  labs(
    x = sprintf("PC1 (%.1f%%)", 100 * pca_variance$variance_explained[1]),
    y = sprintf("PC2 (%.1f%%)", 100 * pca_variance$variance_explained[2])
  ) +
  theme_minimal(base_size = 9)
```

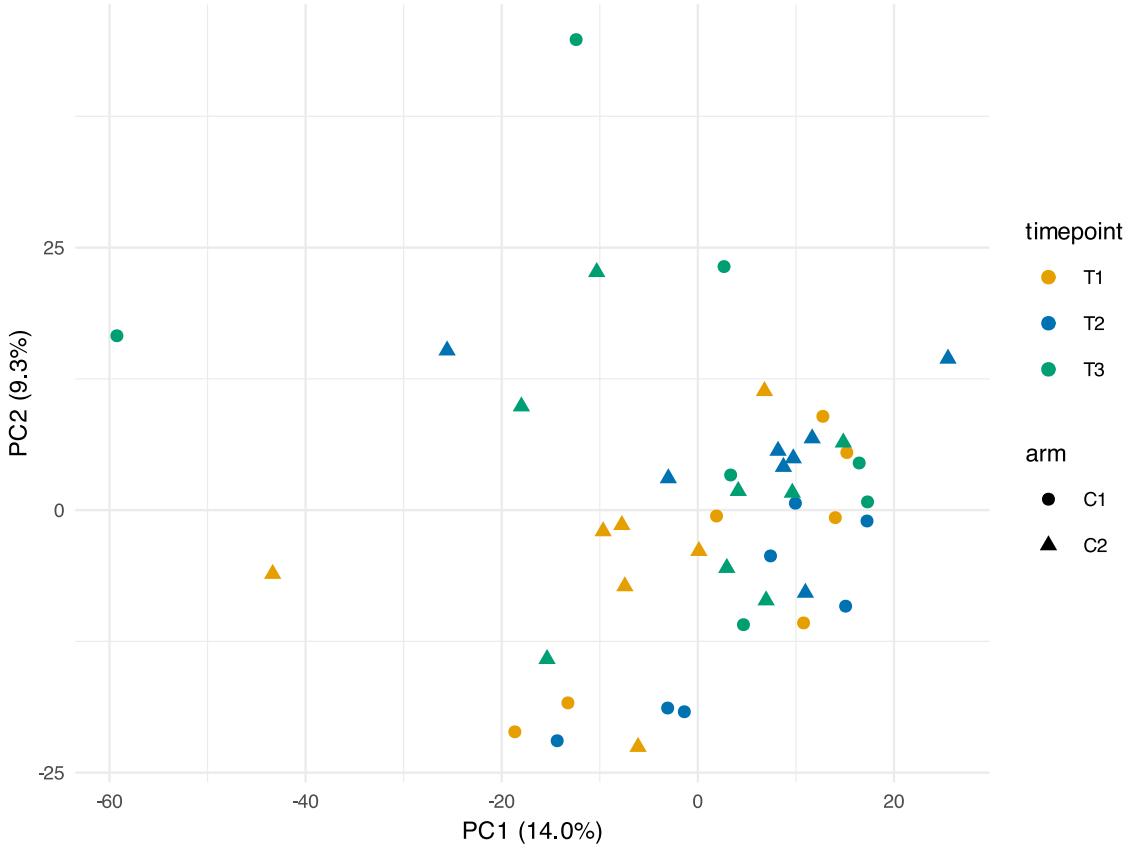


Figure 5: Runs on the blood-corrected matrix, proteins scaled.

Validation

```
stopifnot(
  nrow(results) == nrow(proteins$E) * length(contrast_names),
  all(results$fdr >= results$p),
  all(results$logFC_lower <= results$logFC & results$logFC <= results$logFC_upper),
  !anyNA(results$p)
)
```

Summary

```
time_all <- global_tests |>
  filter(data == "proteome", term == "time", window == "all", arms == "both")
group_mean <- filter(global_tests, term == "arm", window == "subject mean")
```

1900 proteins were tested in each of the nine contrasts; 4 calls at BH < 0.05 across all nine (lowest BH 0.015). Proteins at p < 0.05 run from 52 to 268 per contrast; chance alone gives about 95. Time explains 6.6% of the distance between runs (PERMANOVA p = 0.0013); group, on one profile per person, 6.4% (p = 0.64). The median protein's detectable training interaction is 0.93 log2 units.

Save

`02_fit.rds` holds the fit, design and limpa's matrix; the set and module contrasts read it.

```
out <- here(data_dir)
dir.create(out, recursive = TRUE, showWarnings = FALSE)
saveRDS(
  list(
    eBayes_fit = eb, design = dal$design, metadata = meta, data = proteins$E,
    annotation = dal$annotation, proteins = proteins
  ),
  file.path(out, "02_fit.rds"),
  compress = "xz"
)
write_workbook(
  list(
    protein_by_contrast = results |>
      mutate(n_obs = n_obs[uniprot_id]) |>
      select(
        contrast, uniprot_id, gene, n_obs, logFC, logFC_lower, logFC_upper, t, p, fdr, qvalue
      ) |>
      arrange(match(contrast, contrast_names), p),
    contrast_summary = contrast_summary,
    null_calibration = null_calibration,
    detectable_effect = tibble(
      contrast = "Training_Interaction", quantile = c(0.5, 0.9),
      log2_fc = round(quantile(detectable, c(0.5, 0.9), na.rm = TRUE, names = FALSE), 3)
    ),
    fry_concordance = fry_concordance,
    set_trajectory = set_trajectory,
    unadjusted = unadjusted,
    pca = pca_scores,
    pca_variance = pca_variance,
    permanova = global_tests
  ),
  c(
    paste(
      "One row per contrast and protein: logFC with its 95% CI, t, p, BH and Storey q, and
the",
      "samples observed."
    ),
    "Tested, p < 0.05 and BH counts per contrast, with its role.",
    "propTrueNull, median |t| and share of p below 0.2 per contrast.",
    "log2 effect the primary contrast detects at 80% power, median and 90th percentile.",
    paste(
      "fry: one arm's responders tested in the other arm's contrast, and training sets on
each",
      "arm's acute contrast; with mixed p, mean log2 FC in both contrasts and sign kept."
    ),
    "Training p < 0.05 sets: mean change from each subject's T1 (and SE) per arm and time
point.",
    "logFC and p per protein and contrast before blood adjustment, for sensitivity.",
```

```
    "PC1 and PC2 per sample, from the complete blood-corrected matrix with proteins scaled.",
    "Share of variance on the first four components.",
    paste(
      "PERMANOVA (9,999 permutations): proteome time per window and arm, arm per time point;",
      "phenotype time."
    )
  ),
  file.path(out, "02_protein_contrasts.xlsx"),
  inputs, paths
)
```